# Real-Time Strategy Game AI Opponent using Minimax with Alpha-Beta Pruning

## Artificial Intelligence Project

using Python Programming Language


Guided by : Karthikeyan.L

Presented By :

RICHARD BENSON

RAJ.B.A-1925260077

NITHISH.D.M-192524155

SK ABDUL SHUKUR - 192311390

ATHISH E - 192371015

# Introduction

- Develop an AI opponent for a turn-based strategy game using Python.
- Represent the game as states, legal actions, and transitions.
- Use Minimax to select moves that maximize the AI’s outcome against an opponent.
- Apply Alpha-Beta pruning and move ordering to reduce unnecessary search.
- Use heuristic evaluation and iterative deepening to make decisions within a time limit.

# Literature Survey

| S. No. | Author | Year | Title | Key Idea |
|---|---|---|---|---|
| 1 | John von Neumann | 1944 | Theory of Games and Economic Behavior | Foundation of minimax reasoning in competitive decision-making. |
| 2 | Knuth & Moore | 1975 | An Analysis of Alpha-Beta Pruning | Explains pruning that preserves Minimax results while reducing search. |
| 3 | Richard Korf | 1985 | Depth-First Iterative-Deepening | Shows iterative deepening as a practical search strategy with limited memory. |
| 4 | Russell & Norvig | 2021 | Artificial Intelligence: A Modern Approach | Covers adversarial search, evaluation functions, and game-playing agents. |
| 5 | Python Software Foundation | Current | Python Documentation | Reference for implementing and testing the project in Python. |

# Problem Statement

A strategy-game AI must choose strong moves while considering the opponent's possible responses.

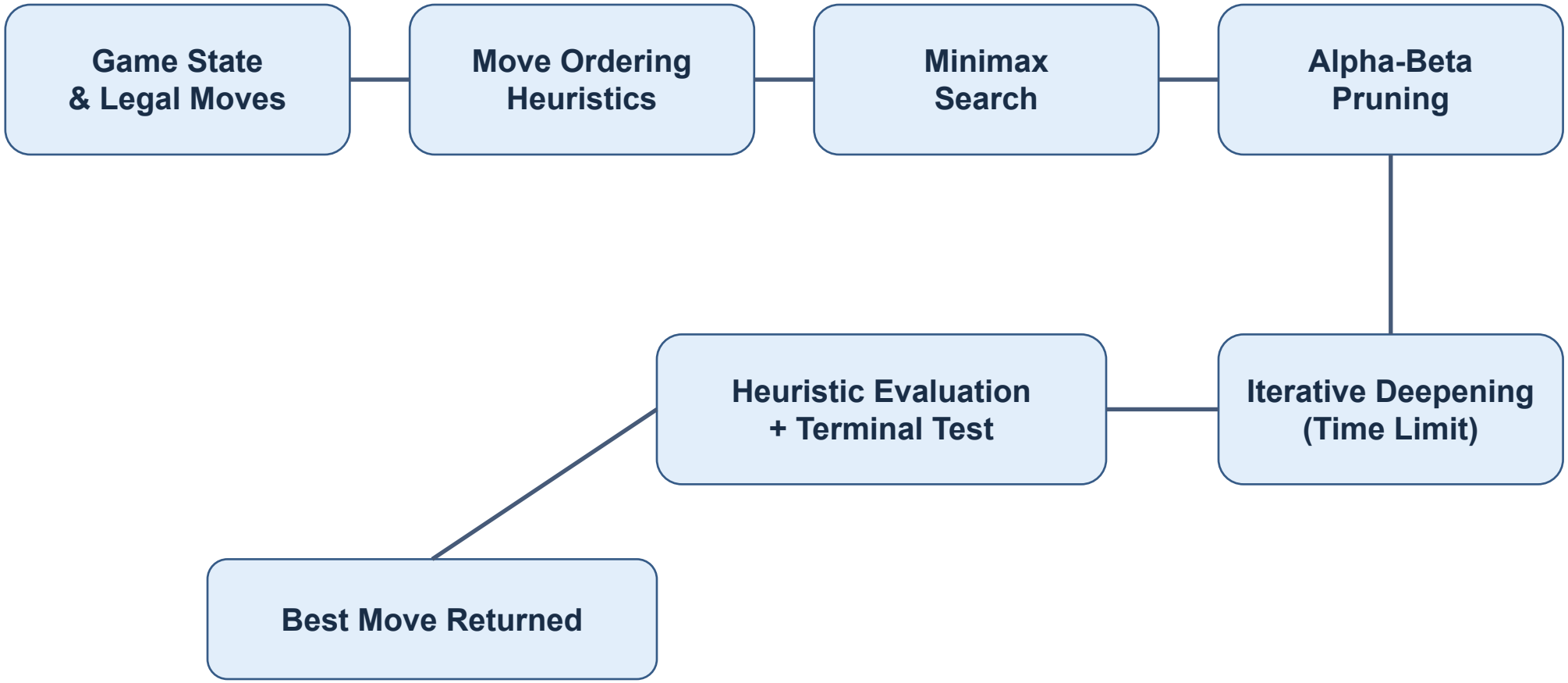
- Searching every possible future move is computationally expensive as the game tree grows.
- A fixed search depth may miss important tactics or take too long on complex positions.
- The system needs efficient state modelling, move generation, evaluation, and bounded search.
- The goal is to produce a competitive move before the decision time expires.

# Objectives

Model the game environment and represent each game state.

- Generate all legal moves available from a given state.
- Implement a depth-limited Minimax decision engine in Python.
- Optimize search using Alpha-Beta pruning and move-ordering heuristics.
- Design a heuristic evaluation function for non-terminal states.
- Use time-bounded iterative deepening to return the best move found.

# System Architecture



# Module 1 – Game Environment & State Representation

- Represent board, active player, legal actions, and terminal conditions.
- Create a state-transition function that applies a move and returns the next state.
- Generate legal moves while preventing invalid actions.
- Keep game rules separate from the search engine for easier testing.
- Provide a consistent state interface for evaluation and search.

# Module 1 – Output: Game State

| | | | | |
|---|---|---|---|---|
| A | | B | | |
| | A | | B | |
| | | A | | B |
| B | | | A | |
| | B | | | A |

Current Player: AI (A)

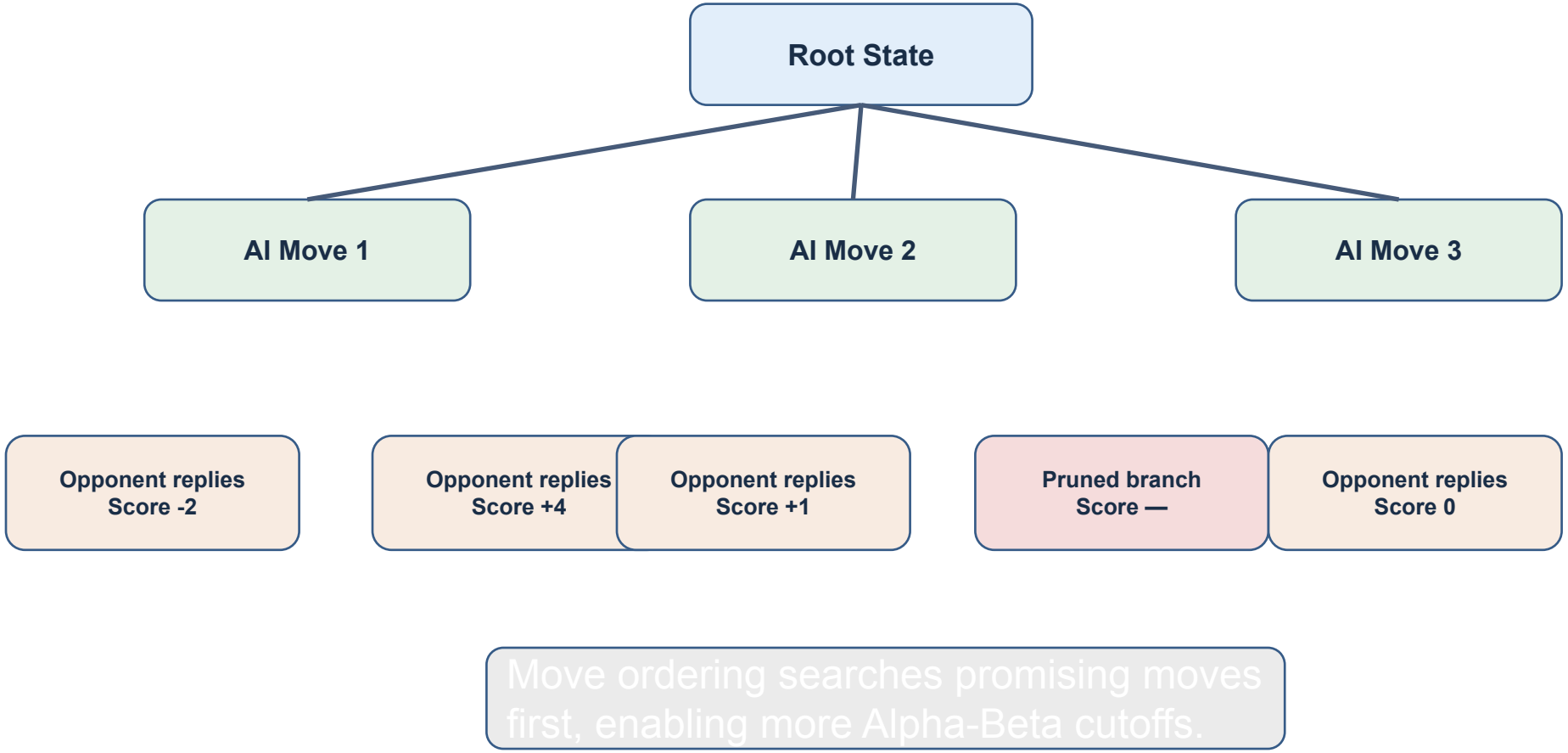
Legal Moves: generated

Next State: apply move

# Module 2 – Depth-Limited Minimax Decision Engine

- Implement depth-limited Minimax with alternating maximizing and minimizing turns.
- Generate possible future states from each legal move.
- Evaluate leaf states when the selected search depth is reached.
- Choose the move with the best backed-up Minimax score.
- Use terminal win, loss, or draw scores when the game ends.

# Module 3 – Output: Alpha-Beta Pruning & Move Ordering

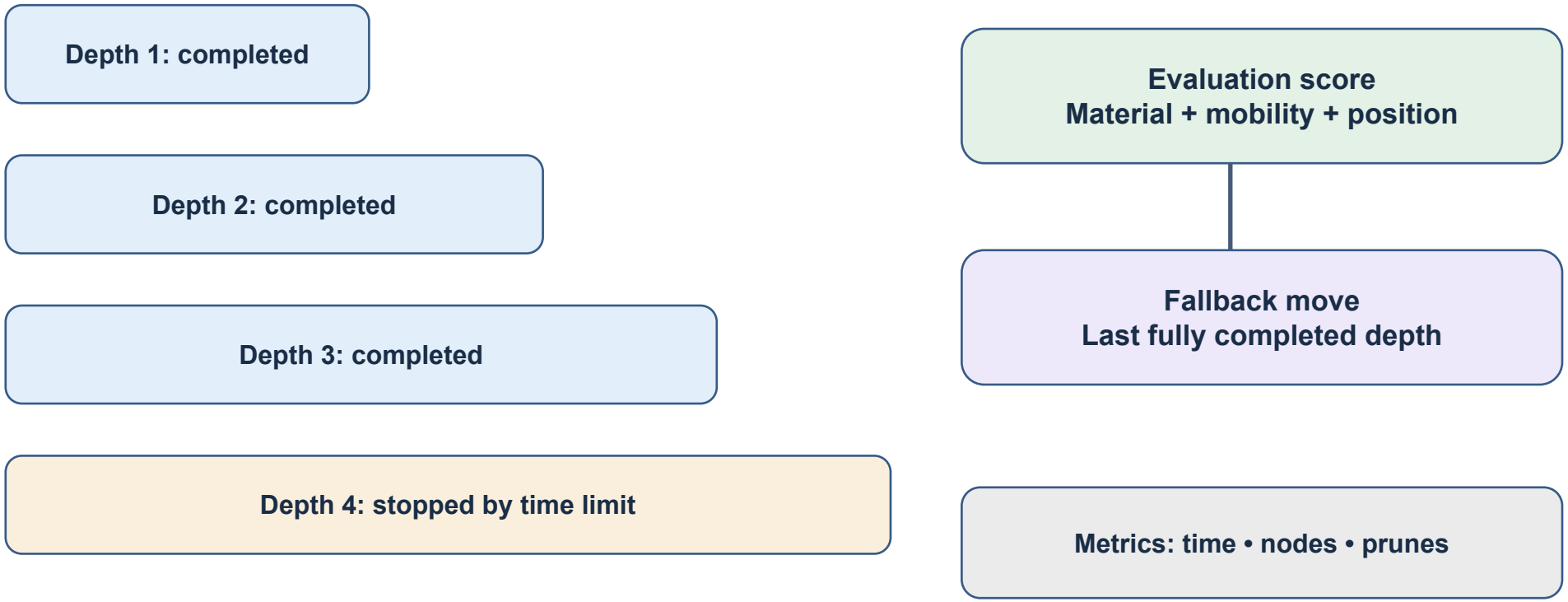



Move ordering searches promising moves first, enabling more Alpha-Beta cutoffs.

# Module 4 – Evaluation Function & Time-Bounded Iterative Deepening

- Design a heuristic evaluation function for non-terminal game states.
- Combine material or territory advantage, mobility, threats, and positional control.
- Run iterative deepening at depths 1, 2, 3, and beyond within a fixed time budget.
- Keep the best move from the last fully completed depth if the time limit is reached.
- Measure search time, nodes explored, and pruning efficiency in Python.

# Module 4 – Output: Evaluation & Time-Bounded Iterative Deepening



# Future Scope

- Add stronger domain-specific evaluation features and tactical pattern recognition.
- Introduce transposition tables and state hashing to reuse repeated positions.
- Tune move ordering with killer moves, history heuristics, or principal variation search.
- Create a graphical interface and adjustable difficulty levels.
- Benchmark playing strength, decision time, and nodes searched across test scenarios.
- SDG 9 – Industry, Innovation and Infrastructure: demonstrate how efficient AI search, algorithmic optimization, and Python-based innovation support reliable intelligent systems.

# Conclusion

The project builds a strategic AI opponent using Python.

- State-space modelling and move generation provide the foundation for search.
- Minimax selects moves by considering both the AI and opponent decisions.
- Alpha-Beta pruning and move ordering reduce search effort.
- Heuristic evaluation and iterative deepening support practical, time-bounded decisions.
- The project supports SDG 9 by applying innovation and efficient computational methods.

# References


Russell, S. and Norvig, P. Artificial Intelligence: A Modern Approach. Pearson.

- Knuth, D. E. and Moore, R. W. “An Analysis of Alpha-Beta Pruning.” Artificial Intelligence, 1975.
- Korf, R. E. “Depth-First Iterative-Deepening: An Optimal Admissible Tree Search.” Artificial Intelligence, 1985.
- Python Software Foundation. Python Documentation: https://docs.python.org/3/
- United Nations. Sustainable Development Goal 9: Industry, Innovation and Infrastructure.


Thank You